FUNKYORM · EXECUTIVE OVERVIEW

Every query, attributable to a person

The audit context: per-request identity, row-level security and audit attribution in the data layer, for .NET teams that answer to auditors.

Funcular Labs · FunkyORM 3.12 (Funcular.Data.Orm on NuGet) · October 2026

Shared identities hide who did what

Modern apps reach the database as one identity: a managed identity, a service account, a pooled login. That’s sound security practice, and it removes the person from the database’s own records.

WHAT THE EXAMINER ASKS

“Which person read this record, under which role, as part of which request?”

WHAT THE DATABASE RECORDED

```
login: app-service-identity
SELECT … FROM record
```

Who it was, and why they were allowed, lived only in application code.

Regulated work needs the person in the record

Individual accountability

Activity tied to a unique user, in logs you can examine.

HIPAA §164.312(a)(2)(i) and §164.312(b) · PCI DSS Requirement 10

Least privilege

Access limited to what each person needs, enforced close to the data.

HIPAA §164.312(a)(1) · PCI DSS Requirement 7

Evidence on demand

Records an auditor can read without reconstructing application internals.

SOX IT general controls over access

FunkyORM is a building block for these controls, not a certification. Your auditors decide what satisfies them.

The pool drops context set outside the ORM

Database session context belongs to one connection, and the pool resets it when the connection is returned. Context set once per request never reaches the connection the next query runs on.

SET ONCE, OUTSIDE THE ORM

1. Middleware sets the user on one connection
2. The query runs on another pooled connection
3. The database sees only the service account

PRIMED BY FUNKYORM

1. Middleware sets the request’s context in memory
2. FunkyORM primes each command’s own connection first
3. The database sees the person on every command, in a transaction or not

How it works: one step per layer

STEP 1

Sign-in

Your identity provider authenticates the person.

STEP 2

Middleware

Builds the request’s context: user id, role snapshot, request id.

STEP 3

FunkyORM

Primes it onto each connection before the first command, in one parameterized batch.

STEP 4

Database

Policies filter rows by it; the audit log records who ran each statement.

Queries don’t change: the same LINQ and the same repositories, with the person attached.

You decide what rides along

```
accessor.Set(new FunkyAuditContext
{
    Entries = new[]
    {
        new SessionContextEntry("app.user_id", userId),
        new SessionContextEntry("app.roles",   roleIds),
        new SessionContextEntry("app.request", requestId),
    },
    AuditSubjectId     = userId,
    AuditCorrelationId = requestId,
});
```

- The keys and what they mean are yours: user, role snapshot, tenant, purpose of use.
- A role snapshot records what the person was allowed at that moment, readable by policies, triggers and audit tables.
- Values are opaque strings: FunkyORM doesn’t interpret them.
- Identifiers, never names: no email or other personal data.

Architects make it mandatory

Fail closed

A strict provider throws before any SQL is sent when a command has no context. Use it for regulated repositories.

Decided once

Health checks and app settings get a lenient provider. The choice sits in your ORM factory, not in every call.

Tamper-resistant

On SQL Server the keys are set read-only, so code in the app can’t swap the identity mid-session.

Schema discovery runs under a system scope, so a strict provider still starts cleanly. A strict provider pointed at SQLite, which has no session context, refuses to run.

The statement names who ran it

Example, as the database’s audit log captures it

```
/* funky:audit sub=u-48213 corr=req-7f3k2q */
SELECT … FROM record WHERE …
```

Fits your audit tooling

Azure SQL Auditing, pgaudit or MySQL’s audit log attribute each statement, even when they capture parameters only.

Injection-proof

Only letters, digits and . _ : - up to 128 characters. Anything else throws; nothing is silently stripped.

No extra round trip

It’s part of the command text. Stored procedures carry none, but their connection is still primed.

The database decides which rows each person sees

On SQL Server and PostgreSQL, a row-level security policy reads the primed keys. Reads are filtered, and a block predicate stops writes to other people’s rows.

An application bug that forgets a WHERE clause still returns only that person’s rows.

You write and test the policy; FunkyORM supplies the identity it checks.

```
CREATE FUNCTION dbo.fn_rls(@owner_id nvarchar(64))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS allowed
  WHERE @owner_id =
    CAST(SESSION_CONTEXT(N'app.user_id') AS nvarchar(64));

CREATE SECURITY POLICY dbo.record_rls
  ADD FILTER PREDICATE dbo.fn_rls(owner_id) ON dbo.record,
  ADD BLOCK  PREDICATE dbo.fn_rls(owner_id) ON dbo.record
  WITH (STATE = ON);
```

Capability follows the database

Capability	SQL Server	PostgreSQL	MySQL	SQLite
Row filtering	Yes	Yes	No native RLS	No
Audit attribution	Yes	Yes	Yes	No
Read-only keys	Yes	Emulated	Best effort	n/a
Audit comment	Yes	Yes	Yes	n/a
Strict mode	Enforced	Enforced	Enforced	Refuses to run

PostgreSQL superusers bypass row-level security, so run applications under a regular role. MySQL needs AllowUserVariables=true and supports attribution only.

Proven on every provider, before every release

33

tests dedicated to the feature: 14 unit, 19 against real databases

4

database engines: SQL Server, PostgreSQL, MySQL, SQLite

0

known-vulnerable packages in a fresh install of 3.12.1, at release

- Rows filtered by the primed user, with no leak between pooled operations
- Parallel requests for different users never bleed into each other
- One prime per transaction; every command in it sees the person
- Strict mode throws when the context is missing
- The audit comment arrives end to end
- PostgreSQL policies tested under a regular role, as production runs them

What FunkyORM does, and what stays yours

FUNKYORM

- Primes your context on every connection, in and out of transactions
- Fails closed when a strict provider has no context
- Locks the keys read-only on SQL Server
- Validates the audit comment
- Changes nothing while the feature is off

YOUR TEAM

- Writes and tests the policies, both filter and block
- Chooses the keys, and keeps personal data out of the comment
- Turns on database auditing
- Keeps connection reset on, and runs PostgreSQL without superuser rights
- Treats MySQL as attribution only

Defense in depth: it complements network and database controls; it doesn’t replace them.

A pilot takes about a page of code

STEP 1

Accessor

A five-line class over AsyncLocal.

STEP 2

Register

A singleton in your DI setup.

STEP 3

Middleware

Set the context after sign-in.

STEP 4

Factory

Strict providers for regulated repositories.

STEP 5

Policies

Your row-level security and database auditing.

Suggested pilot

1. Run one service with a lenient provider and confirm the audit log names each person.
2. Make that service’s regulated repositories strict.
3. Add row-level security, starting with one table.

Attribution and least privilege, built into the data layer

- Read the Audit Context Runbook: docs/guides/AUDIT_CONTEXT_RUNBOOK.md
- Install Funcular.Data.Orm 3.12.1 from NuGet
- Pilot one service in lenient mode, then make it strict

github.com/FuncularLabs/Funcular.FunkyOrm